

We reserve the fourth step—constructing an optimal solution from computed information—for the exercises.

The structure of a shortest path

Let's start by characterizing the structure of an optimal solution. Lemma 22.1 tells us that all subpaths of a shortest path are shortest paths. Consider a shortest path p from vertex i to vertex j , and suppose that p contains at most r edges. Assuming that there are no negative-weight cycles, r is finite. If $i = j$, then p has weight 0 and no edges. If vertices i and j are distinct, then decompose path p into $i \xrightarrow{p'} k \rightarrow j$, where path p' now contains at most $r - 1$ edges. Lemma 22.1 says that p' is a shortest path from i to k , and so $\delta(i, j) = \delta(i, k) + w_{kj}$.

A recursive solution to the all-pairs shortest-paths problem

Now, let $l_{ij}^{(r)}$ be the minimum weight of any path from vertex i to vertex j that contains at most r edges. When $r = 0$, there is a shortest path from i to j with no edges if and only if $i = j$, yielding

$$l_{ij}^{(0)} = \begin{cases} 0 & \text{if } i = j, \\ \infty & \text{if } i \neq j. \end{cases} \quad (23.2)$$

For $r \geq 1$, one way to achieve a minimum-weight path from i to j with at most r edges is by taking a path containing at most $r - 1$ edges, so that $l_{ij}^{(r)} = l_{ij}^{(r-1)}$. Another way is by taking a path of at most $r - 1$ edges from i to some vertex k and then taking the edge (k, j) , so that $l_{ij}^{(r)} = l_{ik}^{(r-1)} + w_{kj}$. Therefore, to examine paths from i to j consisting of at most r edges, try all possible predecessors k of j , giving the recursive definition

$$\begin{aligned} l_{ij}^{(r)} &= \min \left\{ l_{ij}^{(r-1)}, \min \{ l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n \} \right\} \\ &= \min \{ l_{ik}^{(r-1)} + w_{kj} : 1 \leq k \leq n \}. \end{aligned} \quad (23.3)$$

The last equality follows from the observation that $w_{jj} = 0$ for all j .

What are the actual shortest-path weights $\delta(i, j)$? If the graph contains no negative-weight cycles, then whenever $\delta(i, j) < \infty$, there is a shortest path from vertex i to vertex j that is simple. (A path p from i to j that is not simple contains a cycle. Since each cycle's weight is nonnegative, removing all cycles from the path leaves a simple path with weight no greater than p 's weight.) Because any simple path contains at most $n - 1$ edges, a path from vertex i to vertex j with more than $n - 1$ edges cannot have lower weight than a shortest path from i to j . The actual shortest-path weights are therefore given by

$$\delta(i, j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \dots \quad (23.4)$$

Computing the shortest-path weights bottom up

Taking as input the matrix $W = (w_{ij})$, let's see how to compute a series of matrices $L^{(0)}, L^{(1)}, \dots, L^{(n-1)}$, where $L^{(r)} = (l_{ij}^{(r)})$ for $r = 0, 1, \dots, n-1$. The initial matrix is $L^{(0)}$ given by equation (23.2). The final matrix $L^{(n-1)}$ contains the actual shortest-path weights.

The heart of the algorithm is the procedure **EXTEND-SHORTEST-PATHS**, which implements equation (23.3) for all i and j . The four inputs are the matrix $L^{(r-1)}$ computed so far; the edge-weight matrix W ; the output matrix $L^{(r)}$, which will hold the computed result and whose elements are all initialized to ∞ before invoking the procedure; and the number n of vertices. The superscripts r and $r-1$ help to make the correspondence of the pseudocode with equation (23.3) plain, but they play no actual role in the pseudocode. The procedure extends the shortest paths computed so far by one more edge, producing the matrix $L^{(r)}$ of shortest-path weights from the matrix $L^{(r-1)}$ computed so far. Its running time is $\Theta(n^3)$ due to the three nested **for** loops.

```

EXTEND-SHORTEST-PATHS( $L^{(r-1)}, W, L^{(r)}, n$ )
1  // Assume that the elements of  $L^{(r)}$  are initialized to  $\infty$ .
2  for  $i = 1$  to  $n$ 
3      for  $j = 1$  to  $n$ 
4          for  $k = 1$  to  $n$ 
5               $l_{ij}^{(r)} = \min \{l_{ij}^{(r)}, l_{ik}^{(r-1)} + w_{kj}\}$ 

```

Let's now understand the relation of this computation to matrix multiplication. Consider how to compute the matrix product $C = A \cdot B$ of two $n \times n$ matrices A and B . The straightforward method used by **MATRIX-MULTIPLY** on page 81 uses a triply nested loop to implement equation (4.1), which we repeat here for convenience:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}, \quad (23.5)$$

for $i, j = 1, 2, \dots, n$. Now make the substitutions

$$\begin{aligned}
l^{(r-1)} &\rightarrow a, \\
w &\rightarrow b, \\
l^{(r)} &\rightarrow c, \\
\min &\rightarrow +, \\
+ &\rightarrow \cdot
\end{aligned}$$

in equation (23.3). You get equation (23.5)! Making these changes to EXTEND-SHORTEST-PATHS, and also replacing ∞ (the identity for min) by 0 (the identity for +), yields the procedure MATRIX-MULTIPLY. We can see that the procedure EXTEND-SHORTEST-PATHS($L^{(r-1)}, W, L^{(r)}, n$) computes the matrix “product” $L^{(r)} = L^{(r-1)} \cdot W$ using this unusual definition of matrix multiplication.²

Thus, we can solve the all-pairs shortest-paths problem by repeatedly multiplying matrices. Each step extends the shortest-path weights computed so far by one more edge using EXTEND-SHORTEST-PATHS($L^{(r-1)}, W, L^{(r)}, n$) to perform the matrix multiplication. Starting with the matrix $L^{(0)}$, we produce the following sequence of $n - 1$ matrices corresponding to powers of W :

$$\begin{aligned}
L^{(1)} &= L^{(0)} \cdot W = W^1, \\
L^{(2)} &= L^{(1)} \cdot W = W^2, \\
L^{(3)} &= L^{(2)} \cdot W = W^3, \\
&\vdots \\
L^{(n-1)} &= L^{(n-2)} \cdot W = W^{n-1}.
\end{aligned}$$

At the end, the matrix $L^{(n-1)} = W^{n-1}$ contains the shortest-path weights.

The procedure SLOW-APSP on the next page computes this sequence in $\Theta(n^4)$ time. The procedure takes the $n \times n$ matrices W and $L^{(0)}$ as inputs, along with n . Figure 23.1 illustrates its operation. The pseudocode uses two $n \times n$ matrices L and M to store powers of W , computing $M = L \cdot W$ on each iteration. Line 2 initializes $L = L^{(0)}$. For each iteration r , line 4 initializes $M = \infty$, where ∞ in this context is a matrix of scalar ∞ values. The r th iteration starts with the invariant $L = L^{(r-1)} = W^{r-1}$. Line 6 computes $M = L \cdot W = L^{(r-1)} \cdot W = W^{r-1} \cdot W = W^r = L^{(r)}$ so that the invariant can be restored for the next iteration by line 7, which sets $L = M$. At the end, the matrix $L = L^{(n-1)} = W^{n-1}$ of shortest-path weights is returned. The assignments to $n \times n$ matrices in lines 2, 4, and 7 implicitly run doubly nested loops that take $\Theta(n^2)$ time for each assignment.

² An algebraic *semiring* contains operations $\oplus$, which is commutative with identity $I_\oplus$, and $\otimes$, with identity $I_\otimes$, where $\otimes$ distributes over $\oplus$ on both the left and right, and where $I_\oplus \otimes x = x \otimes I_\oplus = I_\oplus$ for all x . Standard matrix multiplication, as in MATRIX-MULTIPLY, uses the semiring with $+$ for $\oplus$, $\cdot$ for $\otimes$, 0 for $I_\oplus$, and 1 for $I_\otimes$. The procedure EXTEND-SHORTEST-PATHS uses another semiring, known as the *tropical semiring*, with min for $\oplus$, $+$ for $\otimes$, ∞ for $I_\oplus$, and 0 for $I_\otimes$.

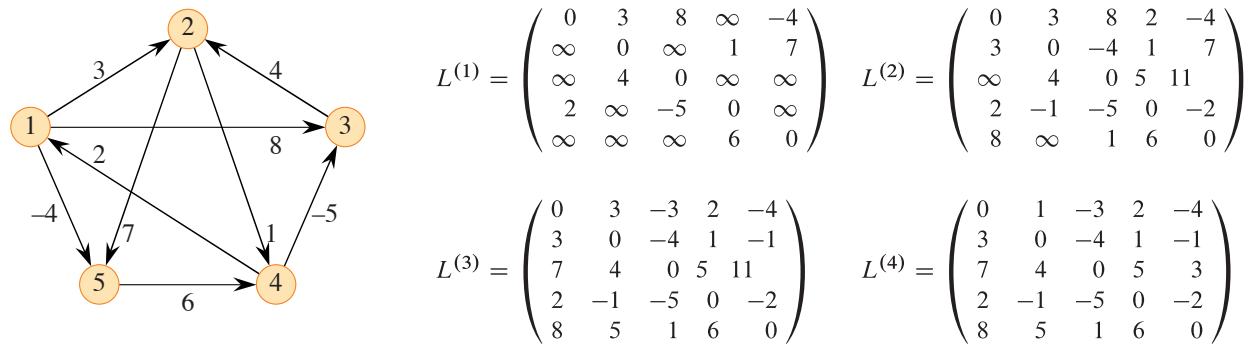


Figure 23.1 A directed graph and the sequence of matrices $L^{(r)}$ computed by SLOW-APSP. You might want to verify that $L^{(5)}$, defined as $L^{(4)} \cdot W$, equals $L^{(4)}$, and thus $L^{(r)} = L^{(4)}$ for all $r \geq 4$.

The $n - 1$ invocations of EXTEND-SHORTEST-PATHS, each of which takes $\Theta(n^3)$ time, dominate the computation, yielding a total running time of $\Theta(n^4)$.

SLOW-APSP($W, L^{(0)}, n$)

```

1  let  $L = (l_{ij})$  and  $M = (m_{ij})$  be new  $n \times n$  matrices
2   $L = L^{(0)}$ 
3  for  $r = 1$  to  $n - 1$ 
4       $M = \infty$  // initialize  $M$ 
5      // Compute the matrix “product”  $M = L \cdot W$ .
6      EXTEND-SHORTEST-PATHS( $L, W, M, n$ )
7       $L = M$ 
8  return  $L$ 
```

Improving the running time

Bear in mind that the goal is not to compute *all* the $L^{(r)}$ matrices: only the matrix $L^{(n-1)}$ matters. Recall that in the absence of negative-weight cycles, equation (23.4) implies $L^{(r)} = L^{(n-1)}$ for all integers $r \geq n - 1$. Just as traditional matrix multiplication is associative, so is matrix multiplication defined by the EXTEND-SHORTEST-PATHS procedure (see Exercise 23.1-4). In fact, we can compute $L^{(n-1)}$ with only $\lceil \lg(n - 1) \rceil$ matrix products by using the technique of *repeated squaring*:

$$\begin{aligned}
L^{(1)} &= W, \\
L^{(2)} &= W^2 = W \cdot W, \\
L^{(4)} &= W^4 = W^2 \cdot W^2, \\
L^{(8)} &= W^8 = W^4 \cdot W^4, \\
&\vdots \\
L^{(2^{\lceil \lg(n-1) \rceil})} &= W^{2^{\lceil \lg(n-1) \rceil}} = W^{2^{\lceil \lg(n-1) \rceil - 1}} \cdot W^{2^{\lceil \lg(n-1) \rceil - 1}}.
\end{aligned}$$

Since $2^{\lceil \lg(n-1) \rceil} \geq n-1$, the final product is $L^{(2^{\lceil \lg(n-1) \rceil})} = L^{(n-1)}$.

The procedure FASTER-APSP implements this idea. It takes just the $n \times n$ matrix W and the size n as inputs. Each iteration of the **while** loop of lines 4–8 starts with the invariant $L = W^r$, which it squares using EXTEND-SHORTEST-PATHS to obtain the matrix $M = L^2 = (W^r)^2 = W^{2r}$. At the end of each iteration, the value of r doubles, and L for the next iteration becomes M , restoring the invariant. Upon exiting the loop when $r \geq n-1$, the procedure returns $L = W^r = L^{(r)} = L^{(n-1)}$ by equation (23.4). As in SLOW-APSP, the assignments to $n \times n$ matrices in lines 2, 5, and 8 implicitly run doubly nested loops, taking $\Theta(n^2)$ time for each assignment.

FASTER-APSP(W, n)

```

1  let  $L$  and  $M$  be new  $n \times n$  matrices
2   $L = W$ 
3   $r = 1$ 
4  while  $r < n - 1$ 
5       $M = \infty$  // initialize  $M$ 
6      EXTEND-SHORTEST-PATHS( $L, L, M, n$ ) // compute  $M = L^2$ 
7       $r = 2r$ 
8       $L = M$  // ready for the next iteration
9  return  $L$ 

```

Because each of the $\lceil \lg(n-1) \rceil$ matrix products takes $\Theta(n^3)$ time, FASTER-APSP runs in $\Theta(n^3 \lg n)$ time. The code is tight, containing no elaborate data structures, and the constant hidden in the Θ -notation is therefore small.

Exercises

23.1-1

Run SLOW-APSP on the weighted, directed graph of Figure 23.2, showing the matrices that result for each iteration of the loop. Then do the same for FASTER-APSP.

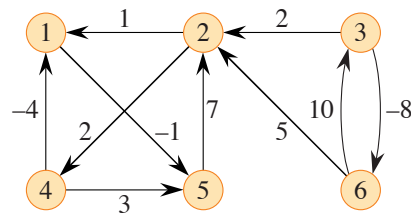


Figure 23.2 A weighted, directed graph for use in Exercises 23.1-1, 23.2-1, and 23.3-1.

23.1-2

Why is it convenient for both SLOW-APSP and FASTER-APSP that $w_{ii} = 0$ for $i = 1, 2, \dots, n$?

23.1-3

What does the matrix

$$L^{(0)} = \begin{pmatrix} 0 & \infty & \infty & \dots & \infty \\ \infty & 0 & \infty & \dots & \infty \\ \infty & \infty & 0 & \dots & \infty \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \infty & \infty & \infty & \dots & 0 \end{pmatrix}$$

used in the shortest-paths algorithms correspond to in regular matrix multiplication?

23.1-4

Show that matrix multiplication defined by EXTEND-SHORTEST-PATHS is associative.

23.1-5

Show how to express the single-source shortest-paths problem as a product of matrices and a vector. Describe how evaluating this product corresponds to a Bellman-Ford-like algorithm (see Section 22.1).

23.1-6

Argue that we don't need the matrix M in SLOW-APSP because by substituting L for M and leaving out the initialization of M , the code still works correctly. (*Hint*: Relate line 5 of EXTEND-SHORTEST-PATHS to RELAX on page 610.) Do we need the matrix M in FASTER-APSP?

23.1-7

Suppose that you also want to compute the vertices on shortest paths in the algorithms of this section. Show how to compute the predecessor matrix Π from the completed matrix L of shortest-path weights in $O(n^3)$ time.

23.1-8

You can also compute the vertices on shortest paths along with computing the shortest-path weights. Define $\pi_{ij}^{(r)}$ as the predecessor of vertex j on any minimum-weight path from vertex i to vertex j that contains at most r edges. Modify the EXTEND-SHORTEST-PATHS and SLOW-APSP procedures to compute the matrices $\Pi^{(1)}, \Pi^{(2)}, \dots, \Pi^{(n-1)}$ as they compute the matrices $L^{(1)}, L^{(2)}, \dots, L^{(n-1)}$.

23.1-9

Modify FASTER-APSP so that it can determine whether the graph contains a negative-weight cycle.

23.1-10

Give an efficient algorithm to find the length (number of edges) of a minimum-length negative-weight cycle in a graph.

23.2 The Floyd-Warshall algorithm

Having already seen one dynamic-programming solution to the all-pairs shortest-paths problem, in this section we'll see another: the *Floyd-Warshall algorithm*, which runs in $\Theta(V^3)$ time. As before, negative-weight edges may be present, but not negative-weight cycles. As in Section 23.1, we develop the algorithm by following the dynamic-programming process. After studying the resulting algorithm, we present a similar method for finding the transitive closure of a directed graph.

The structure of a shortest path

In the Floyd-Warshall algorithm, we characterize the structure of a shortest path differently from how we characterized it in Section 23.1. The Floyd-Warshall algorithm considers the intermediate vertices of a shortest path, where an *intermediate* vertex of a simple path $p = \langle v_1, v_2, \dots, v_l \rangle$ is any vertex of p other than v_1 or v_l , that is, any vertex in the set $\{v_2, v_3, \dots, v_{l-1}\}$.

The Floyd-Warshall algorithm relies on the following observation. Numbering the vertices of G by $V = \{1, 2, \dots, n\}$, take a subset $\{1, 2, \dots, k\}$ of vertices for some $1 \leq k \leq n$. For any pair of vertices $i, j \in V$, consider all paths from i to j whose intermediate vertices are all drawn from $\{1, 2, \dots, k\}$, and let p be a

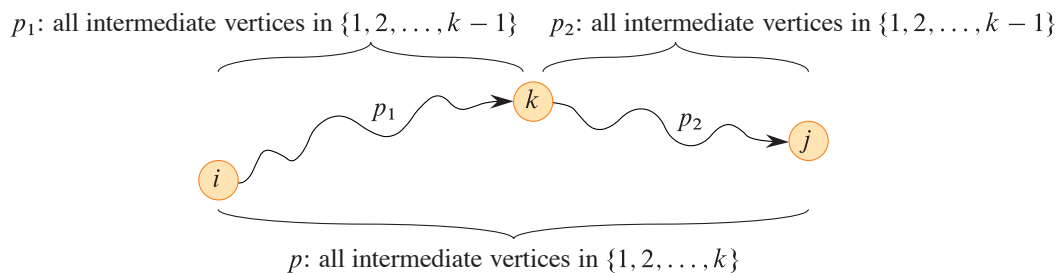


Figure 23.3 Optimal substructure used by the Floyd-Warshall algorithm. Path p is a shortest path from vertex i to vertex j , and k is the highest-numbered intermediate vertex of p . Path p_1 , the portion of path p from vertex i to vertex k , has all intermediate vertices in the set $\{1, 2, \dots, k-1\}$. The same holds for path p_2 from vertex k to vertex j .

minimum-weight path from among them. (Path p is simple.) The Floyd-Warshall algorithm exploits a relationship between path p and shortest paths from i to j with all intermediate vertices in the set $\{1, 2, \dots, k-1\}$. The details of the relationship depend on whether k is an intermediate vertex of path p or not.

- If k is not an intermediate vertex of path p , then all intermediate vertices of path p belong to the set $\{1, 2, \dots, k-1\}$. Thus a shortest path from vertex i to vertex j with all intermediate vertices in the set $\{1, 2, \dots, k-1\}$ is also a shortest path from i to j with all intermediate vertices in the set $\{1, 2, \dots, k\}$.
- If k is an intermediate vertex of path p , then decompose p into $i \xrightarrow{p_1} k \xrightarrow{p_2} j$, as Figure 23.3 illustrates. By Lemma 22.1, p_1 is a shortest path from i to k with all intermediate vertices in the set $\{1, 2, \dots, k\}$. In fact, we can make a slightly stronger statement. Because vertex k is not an *intermediate* vertex of path p_1 , all intermediate vertices of p_1 belong to the set $\{1, 2, \dots, k-1\}$. Therefore p_1 is a shortest path from i to k with all intermediate vertices in the set $\{1, 2, \dots, k-1\}$. Likewise, p_2 is a shortest path from vertex k to vertex j with all intermediate vertices in the set $\{1, 2, \dots, k-1\}$.

A recursive solution to the all-pairs shortest-paths problem

The above observations suggest a recursive formulation of shortest-path estimates that differs from the one in Section 23.1. Let $d_{ij}^{(k)}$ be the weight of a shortest path from vertex i to vertex j for which all intermediate vertices belong to the set $\{1, 2, \dots, k\}$. When $k = 0$, a path from vertex i to vertex j with no intermediate vertex numbered higher than 0 has no intermediate vertices at all. Such a path has at most one edge, and hence $d_{ij}^{(0)} = w_{ij}$. Following the above discussion, define $d_{ij}^{(k)}$ recursively by

$$d_{ij}^{(k)} = \begin{cases} w_{ij} & \text{if } k = 0, \\ \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\} & \text{if } k \geq 1. \end{cases} \quad (23.6)$$

Because for any path, all intermediate vertices belong to the set $\{1, 2, \dots, n\}$, the matrix $D^{(n)} = (d_{ij}^{(n)})$ gives the final answer: $d_{ij}^{(n)} = \delta(i, j)$ for all $i, j \in V$.

Computing the shortest-path weights bottom up

Based on recurrence (23.6), the bottom-up procedure FLOYD-WARSHALL computes the values $d_{ij}^{(k)}$ in order of increasing values of k . Its input is an $n \times n$ matrix W defined as in equation (23.1). The procedure returns the matrix $D^{(n)}$ of shortest-path weights. Figure 23.4 shows the matrices $D^{(k)}$ computed by the Floyd-Warshall algorithm for the graph in Figure 23.1.

```

FLOYD-WARSHALL( $W, n$ )
1   $D^{(0)} = W$ 
2  for  $k = 1$  to  $n$ 
3      let  $D^{(k)} = (d_{ij}^{(k)})$  be a new  $n \times n$  matrix
4      for  $i = 1$  to  $n$ 
5          for  $j = 1$  to  $n$ 
6               $d_{ij}^{(k)} = \min \{d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\}$ 
7  return  $D^{(n)}$ 

```

The running time of the Floyd-Warshall algorithm is determined by the triply nested **for** loops of lines 2–6. Because each execution of line 6 takes $O(1)$ time, the algorithm runs in $\Theta(n^3)$ time. As in the final algorithm in Section 23.1, the code is tight, with no elaborate data structures, and so the constant hidden in the Θ -notation is small. Thus, the Floyd-Warshall algorithm is quite practical for even moderate-sized input graphs.

Constructing a shortest path

There are a variety of different methods for constructing shortest paths in the Floyd-Warshall algorithm. One way is to compute the matrix D of shortest-path weights and then construct the predecessor matrix Π from the D matrix. Exercise 23.1-7 asks you to implement this method so that it runs in $O(n^3)$ time. Given the predecessor matrix Π , the PRINT-ALL-PAIRS-SHORTEST-PATH procedure prints the vertices on a given shortest path.

Alternatively, the predecessor matrix Π can be computed while the algorithm computes the matrices $D^{(0)}, D^{(1)}, \dots, D^{(n)}$. Specifically, compute a sequence of

$$\begin{aligned}
D^{(0)} &= \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(0)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & \text{NIL} & 4 & \text{NIL} & \text{NIL} \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
D^{(1)} &= \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(1)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & \text{NIL} & \text{NIL} \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
D^{(2)} &= \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & 5 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(2)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 1 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
D^{(3)} &= \begin{pmatrix} 0 & 3 & 8 & 4 & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & 5 & 11 \\ 2 & -1 & -5 & 0 & -2 \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} & \Pi^{(3)} &= \begin{pmatrix} \text{NIL} & 1 & 1 & 2 & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 2 & 2 \\ \text{NIL} & 3 & \text{NIL} & 2 & 2 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ \text{NIL} & \text{NIL} & \text{NIL} & 5 & \text{NIL} \end{pmatrix} \\
D^{(4)} &= \begin{pmatrix} 0 & 3 & -1 & 4 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} & \Pi^{(4)} &= \begin{pmatrix} \text{NIL} & 1 & 4 & 2 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix} \\
D^{(5)} &= \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} & \Pi^{(5)} &= \begin{pmatrix} \text{NIL} & 3 & 4 & 5 & 1 \\ 4 & \text{NIL} & 4 & 2 & 1 \\ 4 & 3 & \text{NIL} & 2 & 1 \\ 4 & 3 & 4 & \text{NIL} & 1 \\ 4 & 3 & 4 & 5 & \text{NIL} \end{pmatrix}
\end{aligned}$$

Figure 23.4 The sequence of matrices $D^{(k)}$ and $\Pi^{(k)}$ computed by the Floyd-Warshall algorithm for the graph in Figure 23.1.

matrices $\Pi^{(0)}, \Pi^{(1)}, \dots, \Pi^{(n)}$, where $\Pi = \Pi^{(n)}$ and $\pi_{ij}^{(k)}$ is the predecessor of vertex j on a shortest path from vertex i with all intermediate vertices in the set $\{1, 2, \dots, k\}$.

Here's a recursive formulation of $\pi_{ij}^{(k)}$. When $k = 0$, a shortest path from i to j has no intermediate vertices at all, and so